Basic (a simple version log) — Solution

> Back to the exercise · Hint 1 · Hint 2 · Solution

Story — `version_log_practice.py`: a release gate needs something to compare against, and "which prompt was live last Tuesday?" needs a record. This log is that record — every version saved with its text and time, never overwritten. **If not:** the Build Task's gate would have nothing to check against, and a bad answer couldn't be traced back to the prompt that produced it.

All examples below use a stand-in for Doc13's real test suite, `run_eval_suite(prompt_text)`, so the script runs on its own. In your real Project 5 code, swap it for your actual Doc13 suite import.

Solution

Story: a log you rewrite from scratch each run isn't a history. Load first, then append, and stop loudly if the file is damaged. **If not:** a half-written file would quietly become an empty list, and the next save would wipe every version.

```
# version_log_practice.py
import json
import logging
from datetime import datetime, timezone
from pathlib import Path

logger = logging.getLogger(__name__)
logger.setLevel(logging.INFO)
logger.addHandler(logging.StreamHandler())

class VersionLogCorruptError(Exception):
    pass

def run_eval_suite(prompt_text: str) -> dict:
    if "politely" in prompt_text:
        return {"score": 1.0}
    return {"score": 0.6}

def load_versions(path: str) -> list[dict]:
    if not Path(path).exists():
        return []
    with open(path) as f:
```

```
        try:
            return json.load(f)
        except json.JSONDecodeError:
            # why: a damaged file must stop the save — silently treating
            # it as [] would wipe the whole history on the next write
            raise VersionLogCorruptError(
                f"{path} is not valid JSON — restore it before saving, "
                "or its history will be lost"
            )

def save_version(name: str, prompt_text: str, path: str) -> None:
    # how: load what's already there first, then append — never
    # rebuild the file from scratch
    versions = load_versions(path)
    versions.append({
        "version_name": name,
        "prompt_text": prompt_text,
        "created_at": datetime.now(timezone.utc).isoformat(),
    })
    with open(path, "w") as f:
        json.dump(versions, f, indent=2)
    logger.info(f"Saved version {name} ({len(versions)} in log)")

save_version("v1", "Answer the customer's question.", "versions.json")
save_version(
    "v2",
    "Answer the customer's question politely and cite the source.",
    "versions.json",
)

for version in load_versions("versions.json"):
    result = run_eval_suite(version["prompt_text"])
    print(version["version_name"], result["score"])
```

Expected output:

```
Saved version v1 (1 in log)
Saved version v2 (2 in log)
v1 0.6
v2 1.0
```

(the log lines show only the message, because this handler has no formatter)

And if `versions.json` is damaged — say, half-written when the script was stopped — `save_version` now stops instead of overwriting it. The last line of the traceback:

```
__main__.VersionLogCorruptError: versions.json is not valid JSON — restore it
before saving, or its history will be lost
```

Revision from Doc01 (Intermediate): two habits come back here. First, a named logger, `logging.getLogger(__name__)`, instead of the root logger — so these lines say which module they came from once this code is imported by the release gate. Second, catching one *specific* exception, `json.JSONDecodeError`, and raising a named error with a clear message. This matters a lot here: `save_version` loads, then rewrites the whole file. If a broken file quietly became `[]`, the next save would wipe the whole version history. The named error stops that before any damage is done.